

Relatorio - Sincronizacao em Lista Encadeada com OpenMP

Objetivo

Comparar o tempo de execucao de tres estrategias de sincronizacao para insercao concorrente em listas encadeadas:

- `omp_set_lock/omp_unset_lock`
- `#pragma omp critical` sem nome
- `#pragma omp critical(nome)` com nomes diferentes por lista

Tambem explicar por que usar apenas `critical` com nomes diferentes nao elimina o problema de desempenho neste caso.

Metodologia

Foram implementadas tres versoes separadas:

- `sincronizacao_lock.c`
- `sincronizacao_critical_unnamed.c`
- `sincronizacao_critical_named.c`

Todas as versoes:

- usam 2 threads (`#pragma omp parallel num_threads(2)`)
- executam $N = 50.000.000$ insercoes
- inserem em duas listas (`target_list` aleatoria)
- usam `rand_r` com seed por thread
- contam `total_processed` com `#pragma omp atomic` para manter corretude

Compilacao (macOS + clang + libomp)

```
clang -O2 -Wall -Wextra -std=c11 -Xpreprocessor -fopenmp \
-I/opt/homebrew/opt/libomp/include -L/opt/homebrew/opt/libomp/lib -lomp \
sincronizacao_lock.c -o sincronizacao_lock
```

```
clang -O2 -Wall -Wextra -std=c11 -Xpreprocessor -fopenmp \
-I/opt/homebrew/opt/libomp/include -L/opt/homebrew/opt/libomp/lib -lomp \
sincronizacao_critical_unnamed.c -o sincronizacao_critical_unnamed
```

```
clang -O2 -Wall -Wextra -std=c11 -Xpreprocessor -fopenmp \
-I/opt/homebrew/opt/libomp/include -L/opt/homebrew/opt/libomp/lib -lomp \
sincronizacao_critical_named.c -o sincronizacao_critical_named
```

Execucao

Para cada versao, foram feitas 5 repeticoes com o mesmo N .

Resultados

Amostras de tempo (s):

- lock: 6.304308, 6.221987, 6.194906, 6.189485, 6.254677
- critical sem nome: 9.835537, 10.032035, 9.784474, 9.723183, 9.862554
- critical nomeado: 5.999320, 6.064819, 6.139125, 6.242818, 6.251834

Resumo:

Versao	Media (s)	Min (s)	Max (s)
Lock (omp_set_lock/ omp_unset_lock)	6.233073	6.189485	6.304308
critical sem nome	9.847557	9.723183	10.032035
critical com nomes diferentes	6.139583	5.999320	6.251834

Comparacoes:

$$S_{lock/unnamed} = \frac{9.847557}{6.233073} \approx 1.58$$

$$S_{named/unnamed} = \frac{9.847557}{6.139583} \approx 1.60$$

lock e critical nomeado ficaram muito proximos (diferenca de cerca de 1.50% na media).

Analise

1) critical sem nome

A versao sem nome cria uma secao critica global para todas as insercoes. Assim, mesmo quando as threads querem inserir em listas diferentes, elas ainda serializam no mesmo gargalo. Isso explica o maior tempo medio.

2) critical com nomes diferentes melhora, mas nao “resolve tudo”

Com nomes diferentes (`list_0` e `list_1`), cada lista tem sua propria secao critica. Isso remove a serializacao global e melhora bastante o desempenho em relacao ao critical sem nome.

Porem, isso **nao elimina o problema de contencao**:

- quando duas threads tentam inserir na mesma lista ao mesmo tempo, ainda ha serializacao naquela lista;

- o acesso continua sendo uma regio critica por insercao (granularidade fina), com custo de entrada/saida de sincronizacao em toda iteracao;
- a distribuicao aleatoria para duas listas pode gerar desequilibrios temporarios de carga, mantendo espera entre threads.

Ou seja: nomes diferentes em `critical` reduzem o gargalo global, mas nao removem o custo de sincronizacao frequente nem a contencao por lista.

3) Comparacao com lock

Neste experimento, `critical` nomeado e lock por lista tiveram desempenho semelhante. Ambos resolvem corretude da atualizacao da lista e ambos ainda possuem serializacao local por lista.

Conclusao

A comparacao mostrou:

1. `critical` sem nome foi a pior alternativa de desempenho por serializar todas as insercoes em um unico ponto.
2. `critical` com nomes diferentes reduziu significativamente o tempo, pois separa as secoes criticas por lista.
3. Mesmo assim, `critical` nomeado nao elimina completamente o problema de contencao neste padrao de insercoes frequentes.
4. Lock por lista e `critical` nomeado ficaram praticamente empatados no cenario medido.

Apendice

Apendice A - Arquivos usados

- `sincronizacao_lock.c`
- `sincronizacao_critical_unnamed.c`
- `sincronizacao_critical_named.c`